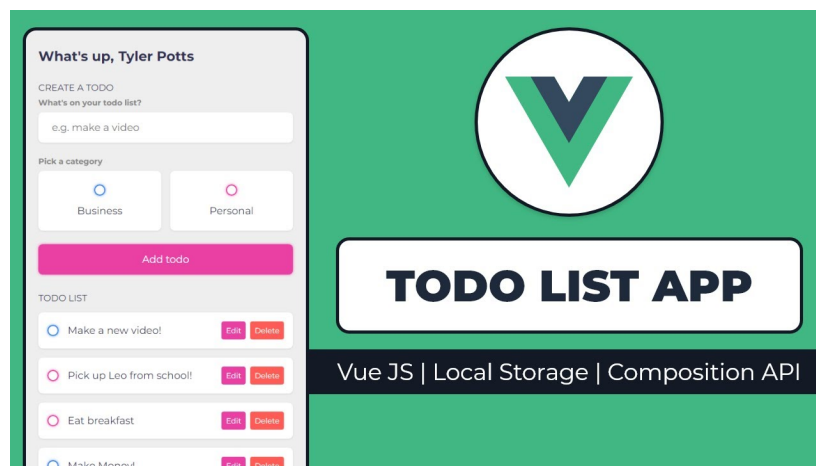




Un projet simple en apparence, mais qui permet d'évaluer de nombreux aspects essentiels du développement frontend : typage, architecture, logique métier, gestion de données, et plus encore. L'objectif est de créer une application de gestion de tâches (To-Do List) en Vue 3 avec TypeScript, en respectant une architecture MVC, et en utilisant le design system DSFR pour l'interface utilisateur.

Table des matières

-  Projet : To-Do List en Vue 3 + TypeScript (MVC + DSFR)
-  Objectif
-  Contraintes techniques
-  Modèle de données (obligatoire)
 - Règles métier
-  Fonctionnalités attendues
 - CRUD des tâches
 - Affichage
 - Filtres (important)
 - Persistance
-  Architecture MVC attendue
-  Organisation recommandée
- Diagramme d'états
-  Documentation attendue
-  Conseils de réalisation
-  Critères d'évaluation

Enoncé

Projet : To-Do List en Vue 3 + TypeScript (MVC + DSFR)

Objectif

Réaliser une application de gestion de tâches en Vue 3 avec TypeScript, en respectant une architecture MVC, avec persistance via localStorage.

L'application devra gérer des tâches enrichies avec des dates, et proposer des filtres avancés.

```

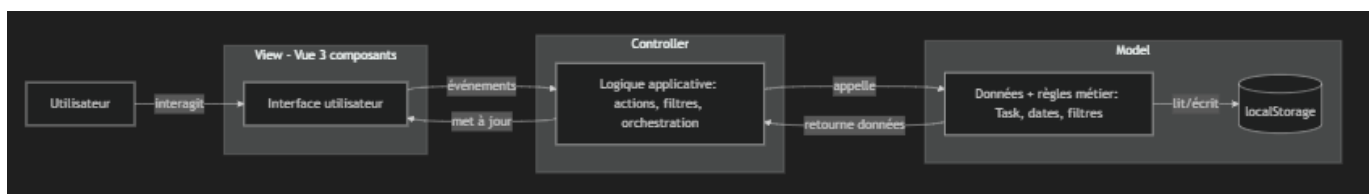
flowchart LR
    User[Utilisateur]

    subgraph View["View - Vue 3 composants"]
        UI[Interface utilisateur]
    end

    subgraph Controller["Controller"]
        C["Logique applicative: actions, filtres, orchestration"]
    end

    subgraph Model["Model"]
        M["Données + règles métier: Task, dates, filtres"]
        LS[(localStorage)]
    end

    User -->|interagit| UI
    UI -->|événements| C
    C -->|appelle| M
    M -->|lit/écrit| LS
    M -->|retourne données| C
    C -->|met à jour| UI
  
```



Contraintes techniques

Framework : Vue 3 (Composition API)

Langage : TypeScript obligatoire

UI : DSFR (classes CSS ou composants) <https://www.systeme-de-design.gouv.fr/>

Stockage : localStorage

Pas de backend

Architecture MVC claire et justifiée

Modèle de données (obligatoire)

Une tâche est définie comme suit :

- interface Task

```
{
  id: number;
  title: string;
  description: string;
  dueDate: string; // ISO string
  completedAt: string | null;
}
```

Règles métier

Une tâche est :

- terminée si `completedAt !== null`
- en retard si `dueDate < aujourd'hui` et non terminée
- à échéance proche si `dueDate` dans les X prochains jours (ex: 3 jours)

```
sequenceDiagram
    actor User as Utilisateur
    participant Form as TaskForm.vue
    participant Controller as task.controller.ts
    participant Model as task.model.ts
    participant Storage as localStorage
    participant App as App.vue

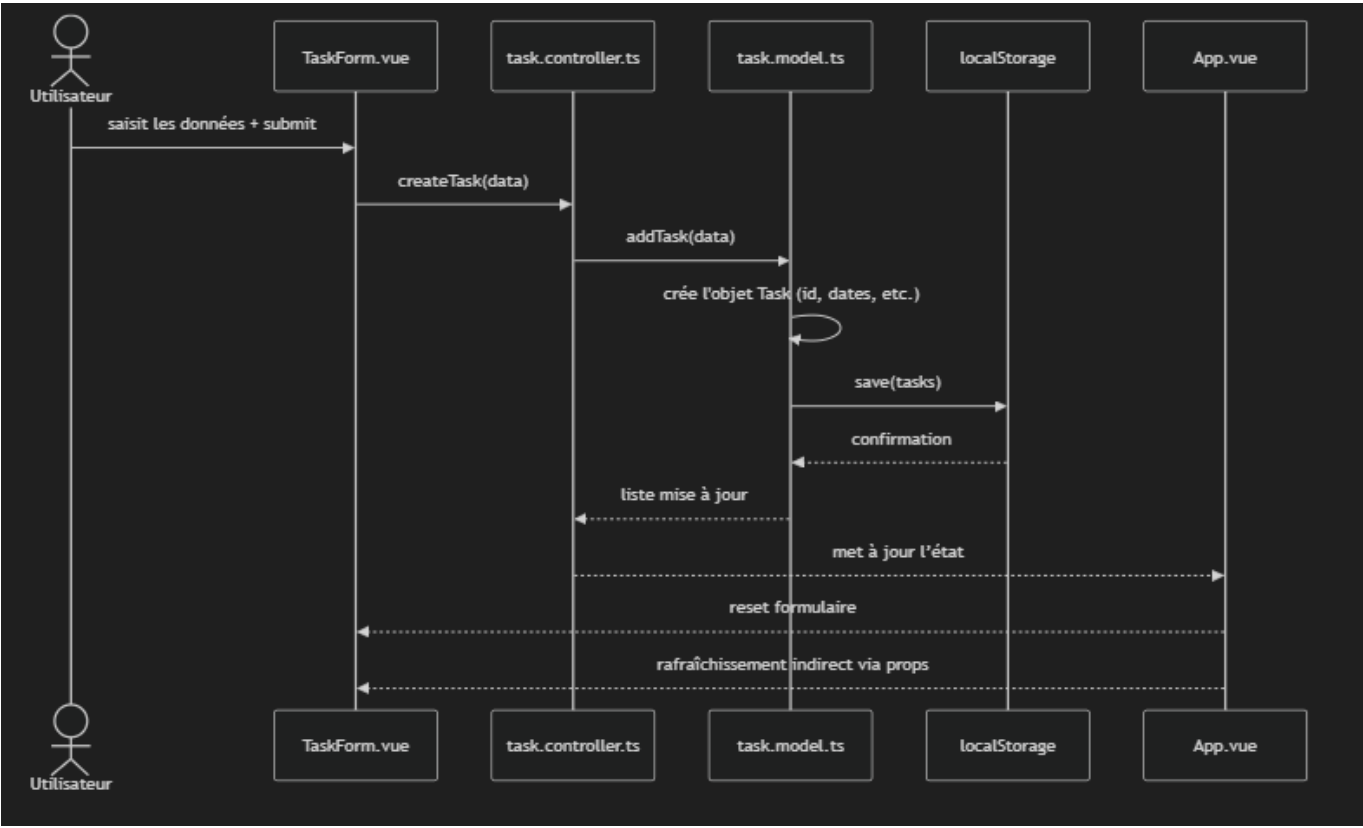
    User->>Form: saisit les données + submit
    Form->>Controller: createTask(data)

    Controller->>Model: addTask(data)
    Model->>Model: crée l'objet Task (id, dates, etc.)

    Model->>Storage: save(tasks)
    Storage-->>Model: confirmation

    Model-->>Controller: liste mise à jour
    Controller-->>App: met à jour l'état

    App-->>Form: reset formulaire
    App-->>Form: rafraîchissement indirect via props
```



Fonctionnalités attendues

CRUD des tâches

Créer une tâche

Supprimer une tâche

Marquer comme terminée (remplit completedAt)

(Optionnel mais recommandé) Modifier une tâche

Affichage

Chaque tâche doit afficher :

- Titre
- Description
- Date d'échéance
- Statut (terminée / en retard / normale)

Filtres (important)

Implémenter au minimum :

- Toutes les tâches
- Tâches terminées
- Tâches en cours
-  Tâches en retard
-  Tâches à échéance proche

Persistance

Sauvegarde automatique dans localStorage

Restauration au chargement

Architecture MVC attendue

Model Responsabilités :

- Définir l'interface Task
- Gérer les données (CRUD)
- Gérer localStorage
- Fournir des fonctions comme :
 - getTasks()
 - addTask(task)
 - deleteTask(id)
 - updateTask(task)
 - getFilteredTasks(filter)

 Le Model ne doit pas dépendre de Vue

Controller Responsabilités :

- Gérer la logique métier :
 - calcul des tâches en retard
 - filtres
- Faire le lien entre Model et View
- Exposer des fonctions utilisées par les composants Vue

View Responsabilités :

- Composants Vue
- Affichage UI (DSFR)
- Interaction utilisateur (clics, formulaires)

Interface (DSFR) Utiliser DSFR pour :

- Formulaire (input, textarea, date)
- Boutons
- Badges / statuts
- Indications visuelles attendues

Tâche en retard → style d'alerte

Échéance proche → style warning

Terminée → style succès ou barré

Organisation recommandée

```
src/  
├── models/  
│   └── task.model.ts  
├── controllers/  
│   └── task.controller.ts  
├── views/  
│   ├── TaskList.vue  
│   ├── TaskItem.vue  
│   ├── TaskForm.vue  
│   └── TaskFilters.vue  
├── types/  
│   └── task.ts  
├── App.vue  
└── main.ts
```


Diagramme d'états

stateDiagram-v2

[*] --> Active

Active --> Completed : terminer()

Completed --> Active : reouvrir()

Active --> Overdue : date dépassée

Overdue --> Completed : terminer()

Overdue --> Active : reouvrir() (si date modifiée)

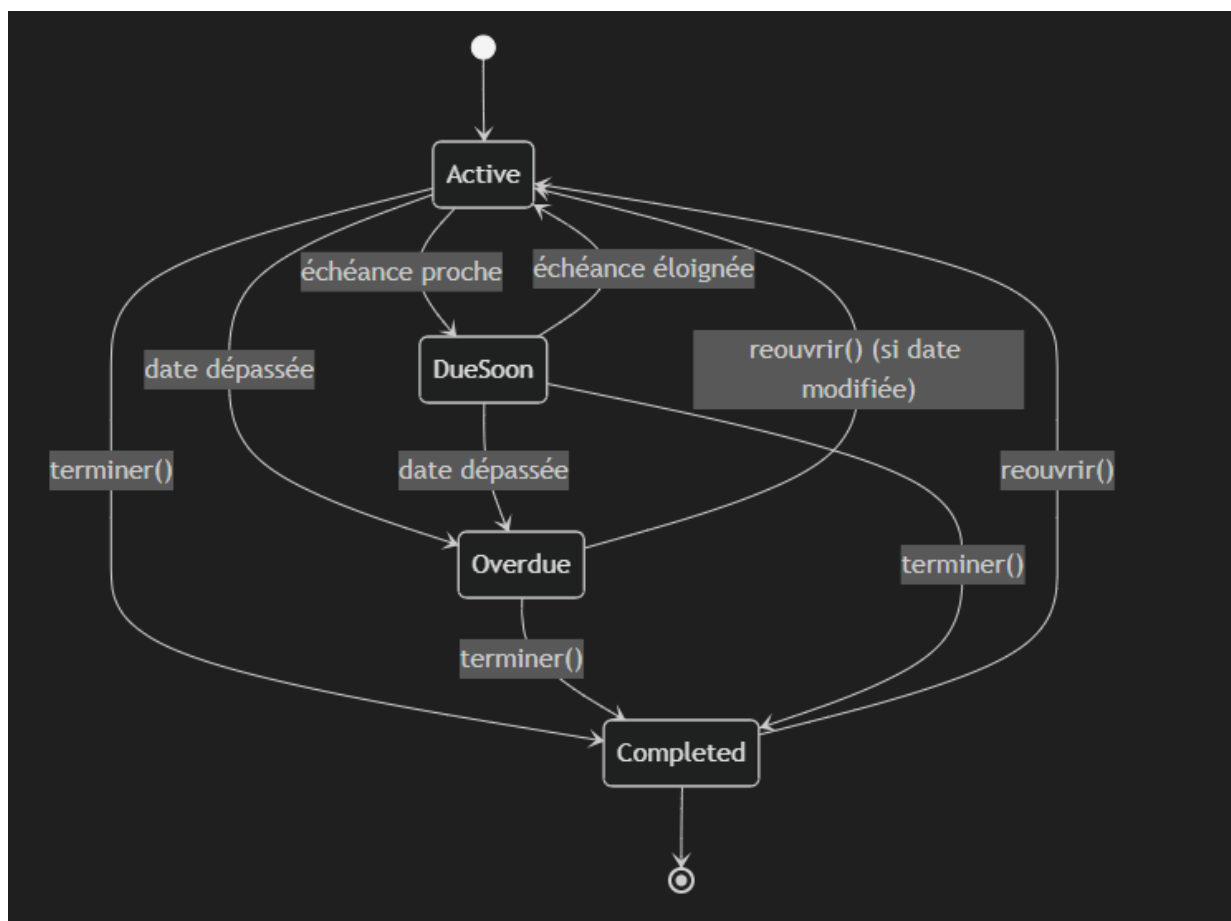
Active --> DueSoon : échéance proche

DueSoon --> Active : échéance éloignée

DueSoon --> Overdue : date dépassée

DueSoon --> Completed : terminer()

Completed --> [*]



Documentation attendue

README.md avec :

1. Présentation
 - Objectif
 - Fonctionnalités
2. Modèle de données
 - Explication de Task
 - Choix des types (string ISO, etc.)
3. Architecture MVC
 - Rôle de chaque couche
 - Schéma simple (même dessiné)
4. Choix techniques
 - Pourquoi TypeScript ?
 - Pourquoi localStorage ?
5. Installation
 - npm install
 - npm run dev

Conseils de réalisation

Ordre recommandé :

- Définir Task + Model
- Implémenter stockage localStorage
- Créer affichage simple
- Ajouter création de tâche
- Ajouter filtres
- Ajouter styles DSFR

☑ Critères d'évaluation

- Typage TypeScript correct
- MVC respecté (séparation claire)
- Fonctionnalités complètes
- Code lisible
- Gestion correcte des dates (⚠ point important)
- Documentation présente

💡 Remarque (important) Le point difficile sera : ⚙ la gestion des dates et des filtres temporels C'est volontaire : cela permet d'évaluer :

- logique métier
- manipulation de données
- rigueur du typage

★ Bonus (si avance)

Tri des tâches (date, statut)

Notifications visuelles

Validation des formulaires

Stockage versionné (migration simple)